

Reviewer Pack — Minimal Rank of Tropical Curves and Communication Complexity...

Entry f0757e53160e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 03:03:42 UTC

Minimal Rank of Tropical Curves and Communication Complexity for XOR

Entry ID: f0757e53160e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 03:03:42 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

{‘text’: ‘For every XOR function with n inputs, there exists a tropical curve over the tropical semiring that computes the XOR, such that the minimal rank of this tropical curve is upper-bounded by a function $f(n) = O(\log^2(n))$. Equivalently, for all XOR functions, the deterministic communication complexity is lower-bounded by $\Omega(\log^2(n))$.’, ‘quantitative relation’: ‘ $E[X(\text{minimal rank})] \leq f(n)$ ’, ‘falsifier’: ‘A counterexample exists if there is an XOR function with n inputs such that the minimal rank of any tropical curve computing it exceeds a function $f(n) = O(\log^2(n))$ and its deterministic communication complexity is less than $\Omega(\log^2(n))$.’}

Rationale (proposer’s reasoning):

{‘text’: ‘This conjecture bridges Tropical Geometry, which has not been widely used in Communication Complexity analysis, with the goal of finding lower bounds for XOR functions. The minimal rank of tropical curves could provide a new avenue for proving communication complexity lower bounds.’, ‘explanation’: ‘Tropical geometry offers a way to represent algebraic structures over tropical semirings, potentially providing a new framework for analyzing communication complexity.’}

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1c449859c37a8bb0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every XOR function, if the minimal rank of its corresponding tropical curve is $\leq O(\log^2(n))$ AND the deterministic communication complexity is $\geq \Omega(\log^2(n))$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry XOR function AND communication complexity - minimal rank tropical curve OR communication complexity lower bound - deterministic communication complexity AND tropical semiring XOR

Top relevant hits considered: - [s2:2511.18099] A Ternary Gamma Semiring Framework for Solving Multi-Objective Network Optimization Problems - [s2:d08d691ee6b02faa316a6cdcab38a593ccfcf771] Mathematical Foundations of Computer Science 1988

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=23.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log2(x):
        if x <= 0:
            return float('-inf')
        return math.log2(x)

    def tropical_add(a, b):
        return max(a, b)

    def tropical_mul(a, b):
        if a == float('-inf') or b == float('-inf'):
            return float('-inf')
        return a + b

    def tropical_neg(a):
        return -a

    def tropical_zero():
        return float('-inf')

    def tropical_one():
```

```

    return 0

def tropical_identity(x):
    return x

def tropical_inverse(x):
    if x == float('-inf'):
        return float('-inf')
    return -x

def tropical_matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[tropical_zero() for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] = tropical_add(C[i][j], tropical_mul(A[i][k], B[k][j]))
    return C

def tropical_matrix_power(M, k):
    if k == 0:
        n = len(M)
        I = [[tropical_zero() for _ in range(n)] for _ in range(n)]
        for i in range(n):
            I[i][i] = tropical_one()
        return I
    elif k % 2 == 1:
        return tropical_matrix_multiply(M, tropical_matrix_power(M, k - 1))
    else:
        half_power = tropical_matrix_power(M, k // 2)
        return tropical_matrix_multiply(half_power, half_power)

def tropical_rank(matrix):
    m = len(matrix)
    n = len(matrix[0])
    rank = 0
    for i in range(min(m, n)):
        if matrix[i][i] != float('-inf'):
            rank += 1
            for j in range(i + 1, m):
                factor = tropical_neg(tropical_divide(matrix[j][i], matrix[i][i]))
                for k in range(n):
                    matrix[j][k] = tropical_add(matrix[j][k], tropical_mul(factor, matrix[i][k]))
    return rank

def tropical_divide(a, b):
    if b == float('-inf'):
        return float('-inf')
    return a - b

def xor_function(x):
    result = 0
    for bit in x:
        result ^= bit

```

```

    return result

def generate_xor_function(n):
    inputs = [random.randint(0, 1) for _ in range(n)]
    outputs = [xor_function(inputs)]
    for _ in range(n - 1):
        new_input = random.randint(0, 1)
        new_output = xor_function(inputs + [new_input])
        inputs.append(new_input)
        outputs.append(new_output)
    return inputs, outputs

def construct_tropical_curve(inputs, outputs):
    n = len(inputs)
    m = n + 1
    A = [[tropical_zero() for _ in range(m)] for _ in range(m)]
    for i in range(n):
        A[i][i] = tropical_one()
        A[n][i] = inputs[i]
    A[n][n] = tropical_one()
    B = [[tropical_zero() for _ in range(1)] for _ in range(m)]
    B[n][0] = outputs[0]
    return A, B

def communication_complexity(inputs):
    n = len(inputs)
    if n == 1:
        return 1
    elif n == 2:
        return 2
    else:
        return n - 1

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
total_communication = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        inputs, outputs = generate_xor_function(n)
        A, B = construct_tropical_curve(inputs, outputs)
        rank = tropical_rank(A)
        comm_complexity = communication_complexity(inputs)
        total_rank += rank
        total_communication += comm_complexity
        instances_tested += 1

mean_rank = total_rank / instances_tested
mean_communication = total_communication / instances_tested

conjecture_holds = mean_rank <= log2(n_values[-1]) ** 2 and mean_communication >= log2(n_values[-1])
counterexample = "" if conjecture_holds else f"Mean rank {mean_rank}, Mean communication {mean_communication}"

return {
    "metric_name": "Communication Complexity",

```

```

        "metric_value": mean_communication,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Mean rank {r['metric_value']}, Mean communication {r['metric_value']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

sted': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}
TRIAL: {'metric_name': 'Communication Complexity', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_holds': False, 'counterexample': 'Mean rank 40.0, Mean communication 38.0'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9e31dc94.py", line 178, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"Mean rank {r['metric_value']}, Mean communication {r['metric_value']}\"")
    ^
NameError: name 'r' is not defined. Did you mean: 're'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for some XOR functions with n inputs, the mean rank of the corresponding tropical curve exceeds $O(\log^2(n))$ and the mea | next: Investigate further to identify the specific XOR function(s) that provide counterexamples and analyze their properties in detail.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15925
2	preregistration	ollama_remote	glm4:latest	0	0	11702
3	novelty	ollama_remote	glm4:latest	0	0	8118
4	novelty	ollama_remote	glm4:latest	0	0	9471
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16923
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12239
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8700
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16315
9	judge	ollama_remote	glm4:latest	0	0	21159

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120551 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f0757e53160e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f0757e53160e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f0757e53160e.tar.gz` (if generated)